Ders 7 Çalışma Özeti

Ders 7 Çalışma Özeti

Genel Konular

- Bağlı listenin (linked list) avantajları, dezavantajları ve kullanım alanları.
- Dairesel bağlı listenin (circular linked list) kullanım alanları.
- Yığın (stack): soyut veri tipi (abstract data type), LIFO (son giren ilk çıkar) mantığı.
- Yığının kullanım alanları: aritmetik ifadeler, özyinelemeli çağrılar, dizgi ters çevirme, geri izleme (backtracking), parantez kontrolü, sayı tabanı dönüşümü, geri al (undo).
- Yığın işlemleri: `initStack`, `isEmpty`, `isFull`, `push`, `pop`, `peek`/`top`, `size`.
- Parantez dengesi kontrolü.
- Dizi ile yığın gerçekleştirme ve tek yönlü bağlı liste ile yığın gerçekleştirme.
- Infix, prefix ve postfix gösterimleri. Infix'ten postfix'e dönüşüm.
- İki yığınla (operand ve operatör) ifade değerlendirme.
- Yığındaki en küçük elemanı yardımcı yığınla (auxiliary stack) takip etme.
- Tek bir diziyi iki yığın olarak kullanma.

Hocanın Özellikle Vurguladığı Kısımlar

- İlk ödev, bir aksilik olmazsa bir sonraki hafta verilecek. Konusu bağlı liste ya da yığın olabilir.
- Yığın ve kuyruk hem diziyle hem bağlı listeyle gerçekleştirilebilir. Hangisinin seçileceği senaryoya bağlıdır.
- Önbellek verimsizliği (cache inefficiency) bağlı listenin en çok dikkat edilmesi gereken dezavantajlarından biridir.
- `push`'tan önce `isFull`, `pop`'tan önce `isEmpty` kontrolü mutlaka yapılmalı. Yoksa segmentation fault oluşabilir.
- `top` değişkeninin 0'dan mı 1'den mi başladığı ve neyi gösterdiği gerçekleştirmeye bağlıdır. Farklı kaynaklarda farklı kabuller görebilirsiniz. Kendi içinde tutarlı olması yeterlidir.
- `push` ve `pop`'un döndürdüğü değer işlemin başarılı olup olmadığını gösterir. Çekilen değer, adresi gönderilen bir değişkene yazılır.
- Bağlı listeyle yığın gerçekleştirirken yeni eleman başa eklenmeli. Böylece çift yönlü listeye gerek kalmaz.
- Bilgisayar infix ifadeyi soldan sağa sırayla hesaplarsa işlem önceliği bozulur. Bu yüzden ifade postfix'e çevrilir.
- Infix-postfix dönüşümünün ve ifade değerlendirmenin akış diyagramını ve kodunu öğrencilerin kendisi yazmalı. Hoca bunu göstermeyecek.
- Yığın ve kuyruk soruları yurt dışındaki teknik iş görüşmelerinde sık soruluyor.

Kısa Tekrar Notları

- Bağlı liste avantajları: dinamik yapı, başta boyut vermeye gerek yok, belleği esnek kullanma, büyük verilere uygun, ölçeklenebilir, ekleme/silmede kaydırma yok.
- Bağlı liste dezavantajları: ek adres bilgisi, doğrudan erişim yok, geriye gitmek zor, ikili arama yapılamaz, gerçekleştirmesi daha karmaşık, blok halinde veri paylaşılamaz, küçük verilerde avantajsız, önbellek verimsizliği.
- Yığın: LIFO. Temel işlemler `push`, `pop`, `peek`, `isFull`, `isEmpty`.
- `peek`, en üstteki elemanı okur ama yığından çıkarmaz.

- Parantez kontrolü: `(` gelince `push`, `)` gelince `pop`. Boş yığında `pop` ya da sonda dolu yığın hatadır.
- Infix: `A + B`. Prefix: `+ A B`. Postfix: `A B +`.
- Dönüşüm kuralları: operand doğrudan çıktıya yazılır. `(` yığına itilir. `)` gelince `(` görülene kadar `pop` yapılıp çıktıya yazılır. Operatör gelince yığının tepesindeki operatörün önceliği büyük ya da eşitse `pop` edilir, sonra gelen operatör itilir. İfade bitince yığın boşaltılır.
- İki yığınla değerlendirme: bir operatör çekilince iki operand çekilir. İlk çekilen ikinci operand, ikinci çekilen birinci operand olur. Sonuç operand yığınına itilir.
- En küçük elemanı takip: gelen eleman yardımcı yığının tepesinden küçükse yardımcı yığına da itilir. Çekilen eleman yardımcı yığının tepesine eşitse oradan da çekilir.
- Bir diziyi iki yığın olarak kullanma: biri baştan (0), diğeri sondan ($n-1$) büyür. Ortada sabit bir sınır konmaz, yığınlar birbirine değene kadar büyüyebilir.

Detaylı Açıklamalar

Bağlı listenin avantajları

- Dinamik bir veri yapısıdır. Dizideki gibi başta boyut vermeye gerek yoktur. Eleman geldikçe bellek kullanılır, bellek boşa harcanmaz.
- Ekleme ve silmede kaydırma gerekmez. Yalnız bağlantı kırılıp yenisi kurulur. Bu işlemlerin çok olduğu durumlarda tercih edilir.
- Belleği esnek kullanır. Arka arkaya bir milyon elemanlık yer yoksa ama farklı bölgelerde yeterli yer varsa bağlı liste bu yerleri kullanabilir.
- Büyük veriler için avantajlıdır ve ölçeklenebilir. Dizide yer bittiğinde genişletmek mümkün değildir.

Bağlı listenin dezavantajları

- Her düğüm için bilginin yanında bir ya da iki adres saklanır. Bu ek bellek demektir.
- Doğrudan (rastgele) erişim (random access) yoktur. Bir elemana ulaşmak için öncekilerin hepsinden geçmek gerekir.
- Geriye doğru gezinmek ek adresleme gerektirir.
- Sıralı olsa bile ikili arama (binary search) uygulanamaz, arama verimliliği düşer.
- Gerçekleştirmesi daha karmaşıktır: kendine referans veren düğümler, yer var mı kontrolü gibi.
- Veri farklı yerlerde durduğu için blok halinde kopyalanıp paylaşılamaz. Elemanlara tek tek ulaşmak gerekir.
- Küçük veri setlerinde bir avantaj sağlamaz.
- Önbellek verimsizliği: dizide birbirini izleyen elemanlar bellekte bir arada olduğu için bir blok halinde önbelleğe alınır. Bağlı listenin düğümleri dağınık olduğu için bu avantaj kullanılamaz.

Bağlı listenin kullanım alanları

- Teorik kullanımlar:
  - Yığın ve kuyruk gerçekleştirme.
  - Ağaç ve graflarda düğümler arası ilişkileri kurma.
  - Dinamik bellek ayırma (dynamic memory allocation).
  - Seyrek matrislerin (sparse matrix) temsili. Örneğin 100 × 100'lük bir matriste yalnız 10 eleman doluysa yalnız bu elemanların satır, sütun ve değerleri saklanır.
  - Büyük tam sayılarla aritmetik işlemler.
  - İşletim sisteminde bellek yönetimi ve süreç zamanlama (process scheduling).
  - Ekleme ve silmenin çok olduğu bütün algoritmalar.
- Gerçek hayat uygulamaları:
  - Müzik çalarda şarkılar, tarayıcıda sayfalar, resim görüntüleyicide resimler arasında ileri-geri gezinme.
  - İşletim sisteminde son kullanılan uygulamalar arasında geçiş.
  - Telefon rehberine yeni kayıt ekleme.

- Dairesel bağlı liste kullanımları: dizi tabanlı kuyruk, medya oynatıcıda başa dönme, hash tablosunda sona gelince başa dönme, navigasyon sistemleri, görev zamanlama (task scheduling).

Yığın (stack)

- Kuyrukta (queue) ilk gelen ilk hizmet alır (FIFO). Yığında tam tersi geçerlidir: son giren ilk çıkar (LIFO), ilk giren son çıkar (FILO).
- Kullanım alanları:
  - Aritmetik ifadelerin infix'ten postfix ya da prefix'e çevrilmesi.
  - Özyinelemeli çağrılar: ilk çağrılan fonksiyon tabanda, son çağrılan tepededir. Sonuçlar yukarıdan aşağı geri aktarılır.
  - Dizgi ters çevirme: ABCD karakterleri sırayla itilip çekilince DCBA elde edilir.
  - Geri izleme: labirentte tıkanınca adım adım geri dönme.
  - Derleyicide parantez dengesi kontrolü.
  - Ondalık sayıları başka sayı tabanlarına çevirme.
  - Metin düzenleyicide geri al (undo): silinen kelimeler yığına itilir, undo yapılınca son silinen ilk geri gelir. Redo da eklenebilir.

Yığın işlemleri

- `push`: yığına eleman koyar.
- `pop`: yığından eleman çıkarır.
- `peek` (ya da `top`): en üstteki elemanın değerini okur ama çıkarmaz.
- `isFull`: yığın dolu mu?
- `isEmpty`: yığın boş mu?
- `size`: yığındaki eleman sayısı.

Parantez dengesi kontrolü

- Bir öğrencinin önerisiyle: her `(` yığına itilir, her `)` için bir `pop` yapılır. Sonda yığın boşsa sözdizimi doğrudur.

Durum	İşlemler	Sonuç
`( ( (`	3 `push`, ifade biter	Yığın dolu kaldı, hata
`)` ile başlıyor	Boş yığında `pop`	Hata
`( ( )`	2 `push`, 1 `pop`	Yığında 1 eleman kaldı, hata
`)` ile başlıyor	Boş yığında `pop`	Hata
`( ( ) )`	2 `push`, 2 `pop`	Yığın boş, hata yok

Diziyle yığın gerçekleştirme

```c
#define MAX 10

struct stack {
    int item[MAX];
    int top;
};

void initStack(struct stack *s)
{
    s->top = 0;
}

int isEmpty(struct stack *s)
{
    if (s->top == 0)
        return 1;
```

```
    return 0;
}

int isFull(struct stack *s)
{
    if (s->top == MAX)
        return 1;
    return 0;
}

int push(struct stack *s, int x)
{
    if (isFull(s))
        return 0;
    s->item[s->top++] = x;
    return 1;
}

int pop(struct stack *s, int *x)
{
    if (isEmpty(s))
        return 0;
    s->top--;
    *x = s->item[s->top];
    return 1;
}

int peek(struct stack *s, int *x)
{
    if (isEmpty(s))
        return 0;
    *x = s->item[s->top - 1];
    return 1;
}
```

- Bu gerçekleştirmede `top`, bir sonraki boş gözü gösterir. `top` aynı zamanda yığındaki eleman sayısıdır.
- `push 5`: `item[0] = 5`, `top = 1`. `pop`: önce `top = 0`, sonra `item[0]` okunur.
- Çekilen eleman dizide fiziksel olarak durur, ama `top` 0 olduğu için yığın boş sayılır. Sonraki `push` 7, 5'in üzerine yazılır.
- `push` ve `pop` iki satır yerine `s->item[s->top++] = x;` gibi tek satırda yazılabilir.
- `peek`, `top`'un kendisini değiştirmez. Yalnız `top - 1` gözündeki değeri okur.
- Bir öğrenci, `top` 0'dan başlıyorsa en fazla 9 olup olamayacağını sordu. Bu gerçekleştirmede `top` eleman sayısını gösterdiği için 10'a kadar çıkar. 10 olduğunda yığın doludur.
- Bir öğrenci, çekilen değeri işaretçi yerine doğrudan `return` ile döndürmeyi sordu. Bu mümkün, ama yığında 0 değeri saklanıyorsa dönen 0'ın hata mı değer mi olduğu ayırt edilemez.

Bağlı listeyle yığın gerçekleştirme

- Bir öğrenci, `pop` sırasında önceki elemanı bilmek için çift yönlü liste gerekip gerekmediğini sordu.
- Cevap: tek yönlü liste yeterli, yeter ki yeni eleman listenin başına eklensin.
  - 3, 4, 2 sırayla itilince liste: `head` → 2 → 4 → 3 → `NULL`.
  - `pop`: `head`'in gösterdiği düğüm (2) çekilir, `head` bir sonraki düğüme (4) bağlanır.
- Yeni eleman sona eklenirse her `push`'ta sona kadar gitmek gerekir. Bu durumda `tail` tutulmalıdır.

Infix, prefix ve postfix

- Infix: operatör iki operandın ortasında (`A + B`). Bizim kullandığımız gösterim.
- Prefix: önce operatör, sonra operandlar (`+ A B`).
- Postfix: önce operandlar, sonra operatör (`A B +`). Bilgisayarlar genelde postfix'i tercih eder.
- Öncelik sırası: parantez, sonra çarpma ve bölme, sonra toplama ve çıkarma. Üs alma bu derste kullanılmıyor.

Infix	Prefix	Postfix
`A + B`	`+ A B`	`A B +`
`A + B * C`	`+ A * B C`	`A B C * +`
`A + B * C + D`	`+ + A * B C D`	`A B C * + D +`
`A + B * C - D / E * F`	`- + A * B C * / D E F`	`A B C * + D E / F * -`

- Örnek: `3 + 5 * 2 / 3 + 7`. Sırayla hesaplanırsa $((3+5)\cdot 2)/3+7 \approx 12{,}33$ çıkar. Öncelikle hesaplanırsa $3 + 10/3 + 7 \approx 13{,}33$ çıkar.
- Başka örnek: `2 * 4 - 8 / 2 + 3`. Doğru sonuç $8-4+3=7$. Sırayla hesaplanırsa 3 çıkar.

Infix'ten postfix'e: parantezli örnek

- İfade: `3 * (5 + 4 / 1) - 2`.

Okunan	İşlem	Yığın	Çıktı
3	Çıktıya yaz		3
*	`push`	*	3
(	`push`	* (	3
5	Çıktıya yaz	* (	3 5
+	`push` (tepede `(` var)	* (+	3 5
4	Çıktıya yaz	* (+	3 5 4
/	`push`	* (+ /	3 5 4
1	Çıktıya yaz	* (+ /	3 5 4 1
)	`(` görülene kadar `pop`	*	3 5 4 1 / +
-	* daha öncelikli, `pop`; sonra `push -`	-	3 5 4 1 / + *
2	Çıktıya yaz	-	3 5 4 1 / + * 2
Son	Yığını boşalt		3 5 4 1 / + * 2 -

İki yığınla ifade değerlendirme

- Derleyiciler dönüşümü ve hesabı aynı anda yapar. Bir operand yığını, bir operatör yığını kullanılır.
- İfade: `5 * (4 / 2 + 3 * 6) - 7`.
  - 5 operand yığınına, * ve (operatör yığınına, 4 operand yığınına, / operatör yığınına, 2 operand yığınına itilir.
  - + gelir. Tepedeki / daha öncelikli olduğu için çekilir. 2 ve 4 çekilir, $4/2 = 2$ operand yığınına itilir. Sonra + itilir.
  - 3, *, 6 itilir.
  -) gelir. (görülene kadar işlenir: $3 \cdot 6 = 18$, sonra $2 + 18 = 20$.
  - - gelir. Tepedeki * daha öncelikli: $5 \cdot 20 = 100$. Sonra - ve 7 itilir.
  - İfade biter: $100 - 7 = 93$. Sonuç operand yığınından çekilir.
- Bir operatör için iki operand çekilir. İlk çekilen sağ operand, ikinci çekilen sol operand olur. Çıkarma ve bölmede bu sıra önemlidir.

Yığındaki en küçük elemanı takip etme

- Problem: yığına `push` ve `pop` yapılırken herhangi bir anda yığındaki en küçük eleman bulunmak isteniyor. Ek bir yardımcı yığın kullanılabilir.

- `push x`: x ana yığına itilir. Yardımcı yığın boşsa ya da x yardımcı yığının tepesinden küçükse x yardımcı yığına da itilir.
- `pop`: ana yığından çekilen eleman yardımcı yığının tepesine eşitse yardımcı yığından da çekilir.
- En küçük eleman her zaman yardımcı yığının tepesindedir.
- Örnek: 3, 5, 1, 2, 4, -5 itiliyor.
  - Ana yığın: 3, 5, 1, 2, 4, -5. Yardımcı yığın: 3, 1, -5. En küçük: -5.
  - `pop` (-5): yardımcı yığının tepesine eşit, oradan da çekilir. En küçük: 1.
  - `pop` (4): eşit değil, yardımcı yığın değişmez. En küçük: 1.
- Bir öğrenci aynı problemin tek yığınla çözülüp çözülemeyeceğini sordu. Hoca bunun da bir çözümü olduğunu, ama bu derste konuşulmayacağını söyledi.

Bir diziyi iki yığın olarak kullanma

- n elemanlı tek bir dizi iki yığın için en verimli nasıl kullanılır?
- Bir öğrencinin fikri: bir yığın 0. gözden, diğeri $n-1$. gözden başlar ve ortaya doğru büyür.
- Bu fikrin kısıtı, sınırı tam ortaya koymaktır. Bir yığın az, diğeri çok kullanıldığında boş kalan yarı kullanılamaz.
- Doğru yaklaşım: iki yığın birbirine değene kadar büyüyebilir. Yalnız `top1` ve `top2`'nin birbirinin üzerinden geçmemesi kontrol edilir.

Ders sonrası notlar

- Hoca bağlı listede ekleme ve silmenin en kötü durumda $O(1)$ olduğunu söyledi. Bu, eklenecek ya da silinecek yerin adresi biliniyorsa doğrudur. Sıralı listede yeri bulmak için önce gezinmek gerekiyorsa işlem $O(n)$ olur. Dizide de kaydırma nedeniyle $O(n)$'dir.
- Parantezsiz örnek olan `3 * 5 + 4 / 1 - 2`'de hoca `-` geldiğinde yalnız `/`'yi çekip `-`'yi `+`'nın üzerine itti. Çıktı `3 5 * 4 1 / 2 - +` oldu, yani $15 + (4 - 2)$. Doğru kural, tepedeki operatörün önceliği gelen operatörle eşit ya da büyük olduğu sürece çekmeye devam etmektir. `+` ile `-` eşit öncelikli ve soldan sağa işlendiği için `+` da çekilmelidir. Doğru postfix: `3 5 * 4 1 / + 2 -`. Bu örnekte iki çıktı da 17 verir, ama `A - B + C` gibi ifadelerde hatalı kural yanlış sonuç üretir.
- Hoca bu örneğin başlığını önce "prefix" yazdığını fark edip "postfix" olarak düzeltti.
- En küçük eleman takibinde yardımcı yığına yalnız "küçükse" itme kuralı, aynı en küçük değer iki kez geldiğinde sorun çıkarır. Örneğin 1, 1 itilir ve bir `pop` yapılırsa yardımcı yığındaki tek 1 de çekilir. Oysa yığında hâlâ 1 vardır. Kural "küçük ya da eşitse it" olmalıdır.
- Bir dizide iki yığında dolu olma kontrolü, gerçekleştirmeye göre `top1 == top2` ya da `top1 + 1 == top2` gibi yazılır. Önemli olan iki yığının aynı göze yazmamasıdır.
- Kaydın 00:25:53 ve 02:09:58 civarındaki "Altyazı M." satırları altyazı hatasıdır, derste söylenmemiştir.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.